

Phase 5: Project Development Phase

Coding & Solution Implementation

1. Overview of Core Implementation

This document presents the complete technical implementation of the ComicCraft AI platform. The system operates as a cohesive pipeline coordinating prompt analysis, structured story outlining, narrative expansion, concurrent image inking, layout building, and multi-page PDF compilation.

2. Core Python Modules

2.1 Gemini Flash Outliner (app/gemini_flash.py)

Responsible for generating a structured 5-panel comic storyline schema based on user inputs.

```
def generate_outline(user_prompt: str) -> List[Dict[str, Any]]:
```

```
"""
```

Generates a 5-panel comic layout based on the user's story idea using Gemini.

Returns:

List: A list of dictionaries, one for each panel.

```
"""
```

prompt = f"""Your task is to generate a strictly formatted JSON array containing 5 panel descriptions for a comic based on the story idea below:

Story: "{user_prompt}"

Each JSON object must include:

- "panel" (integer)
- "title" (string)
- "scene_description": "scene description here"
- "image_prompt": "Image prompt for Stable Diffusion"

Output only the JSON array."""

Queries Gemini 1.5 Flash, validates structure, and returns 5 panel dicts

2.2 Gemini Pro Storyteller (app/gemini_pro.py)

Expands panel outlines into full comic dialogue, environmental captions, and narrator text.

```
def generate_story(outline: list) -> str:
```

```
"""
```

Generates a detailed comic story with narration and character dialogue from a list of comic panel outlines using Gemini 1.5 Pro.

```
"""
```

Formats numbered outline and prompts Gemini Pro with comic book writer prompt

Returns formatted markdown containing **Panel X: Title**, **CAPTION:**, *NA

2.3 Illustration Generator (app/image_generator.py)

Inks comic-style illustrations from visual scene prompts.

```
def generate_image(prompt: str, filename: Optional[str] = None) -> str:
```

```
"""
```

Creates a comic-style image based on the provided image prompt.
Sanitizes prompt, applies caching, and renders panel illustration into static/panels/.
"""

Employs Diffusion / inking pipeline with procedural Pillow artistic fallback

2.4 Layout Builder (app/layout_builder.py)

Assembles disparate visual and text components into unified panel structures.

def build_comic_layout(image_paths, full_story, outline):

**Splits story by **Panel, extracts dialogue/caption/narration tags,
and binds image file paths with panel metadata into a structured list.**

2.5 Multi-Page PDF Exporter (app/exporters.py)

Compiles the comic into a print-ready PDF using fpdf2.

class ComicPDF(FPDF):

def footer(self):

self.set_y(-12)

self.cell(0, 8, f"ComicCraft - AI Comic Book Creator | Page {self.page_no()}", align="C")

def save_pdf(layout: List[Dict[str, Any]]) -> str:

Formats each panel on exactly 1 dedicated A4 page with TrueType Unicode font

2.6 FastAPI Router (app/routes.py)

Exposes all endpoints and coordinates the end-to-end pipeline:

- GET /: Homepage with interactive form and story presets.
- POST /generate: Form submission handler with concurrent multi-thread inking (ThreadPoolExecutor).
- POST /generate-comic/json: Headless JSON API endpoint.
- GET /download/{filename}: Secure attachment download stream.
- GET /export-success: Post-download celebratory confirmation page.
- GET /test-image: Standalone image generation test route.